Prompt Life v0.28.0 Visual Aid Readiness Report

Generated 2026-06-13T17:11:29.405Z for Prompt Life v0.28.0.

This readiness pass reclassifies all Journey visual aids against a human-readability rubric, applies the six-template strategy, and verifies that no P0/P1 visual remains for small human testing.

39	18	0	6
visual aids scored	P0/P1 before	P0/P1 after	templates used

Summary Of Changes

- Added six canonical visual-aid templates to the style guide.
- Scored all 39 Journey visual aids with a seven-part human-readability rubric.
- Created a full human-test review packet and tester feedback sheet.
- Strengthened `audit:visual-aids` to check templates, captions, alt text, label estimates, generated/mechanism boundaries, P0/P1 status, metadata strings, and screenshot coverage.
- Bumped app/package version to 0.28.0.

P0/P1 Before And After

Priority	Before	After
P0	0	0
P1	18	0
P2	8	17
P3	13	22

Template Distribution

- **Atmospheric Scene**: 9
- **Taxonomy Map**: 6
- **Comparison Board**: 8
- **Mechanism Flow**: 9
- **Probability Bars**: 3

- **Context Tray / Stack**: 4

Visuals Fixed For Testing

- Prompt vs Response
- Text to Tokens
- Relevance Between Tokens
- MLP Feature Workshop
- Transformer Stack
- Hidden State Flow
- Raw Scoreboard
- Score to Probability
- Weighted Token Choice

Temporary But Acceptable For Testing

- Overfitting vs Generalization
- Fine-Tuning Path
- Feature Vector
- Tensor Block
- Learning Modes Matrix
- Media Lane Map
- Storm Front
- Borrowed Flames
- Cost Ledger
- Benefit Tiers
- Accountability Flow
- Better-AI Control Panel
- Full Chain Map

Recommended Later Image 2 Replacements

- Prompt to Prediction
- Broad Pretraining
- Fine-Tuning Path
- Alignment Landscape
- Storm Front
- Borrowed Flames
- Cost Ledger
- Benefit Tiers
- Accountability Flow

Must Remain Coded SVG/HTML

- AI Family Tree
- Rules and Learned Patterns
- Training Loop
- Overfitting vs Generalization
- Forward Pass
- Prompt vs Response
- Text to Tokens
- Token IDs
- Embedding Lookup
- Feature Vector
- Tensor Block
- Relevance Between Tokens
- MLP Feature Workshop
- Transformer Stack
- Hidden State Flow
- Raw Scoreboard
- Score to Probability
- Weighted Token Choice
- Append and Repeat
- Temporary Context Window
- Open-Book Retrieval
- Claim Support Map
- Unsupported Bridge
- Learning Modes Matrix
- Append Or Denoise
- Media Lane Map
- Risk Ledger
- Better-AI Control Panel
- Context Tray
- Full Chain Map

Screenshots

Prompt to Prediction

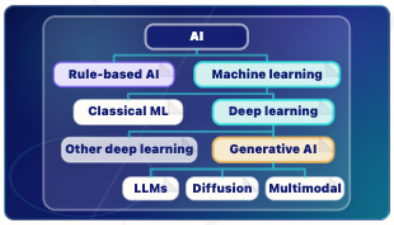
Score, choose, append, repeat

Current context enters the model, learned weights shape next-token probabilities, one token is generated, and the response grows token by token.

- **Context enters** The current prompt and response-so-far become the visible input for this run.
- **Weights shape probabilities** Learned weights, built earlier during training, shape next-token scores.
- **One token is generated** The model chooses one response token from the probability cloud.
- **Response grows** That token is appended, then the model runs again.

Key takeaway:
An LLM turns context into next-token probabilities.

Prompt to Prediction



AI Family Tree

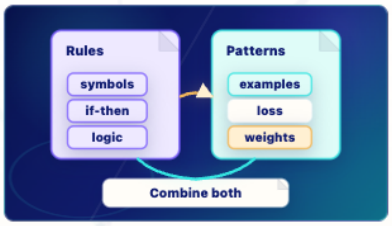
Where LLMs fit

A clean taxonomy tree shows AI as the broad field, machine learning as one branch inside AI, and LLMs as one branch inside generative AI.

- **AI** AI is the broad field.
- **Machine learning** Machine learning systems learn patterns from data.
- **Deep learning** Deep learning is a neural-network branch of machine learning.
- **Generative AI** Generative AI creates new media, such as text, images, audio, code, or video.
- **LLMs** LLMs generate language/code; diffusion and multimodal systems are neighboring generative AI branches.

Key takeaway:
An LLM is one kind of generative AI inside the broader AI family.

AI Family Tree



Rules and Learned Patterns

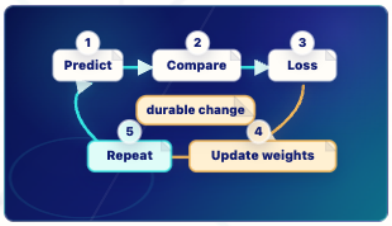
Two traditions, one modern toolkit

Rules-first AI uses symbols and if-then logic; deep learning learns from examples by predicting, measuring loss, and updating weights.

- **Rules** Symbolic systems use explicit if/then logic and symbols.
- **Examples** Loss is the training signal: it measures prediction error so weights can be adjusted.
- **Bridge** Modern systems often combine learned models with rules, retrieval, tools, filters, and policies.

Key takeaway:
Modern AI often blends learned patterns with hand-built rules and tools.

Rules and Learned Patterns



Training Loop

Durable change happens at weight update

Predict, compare, loss, update weights, repeat. Training changes weights.

1. **Predict** The model predicts a target.
2. **Compare** The prediction is compared with the target.
3. **Loss** Loss measures error.
4. **Update weights** Weight updates are the durable-change step.
5. **Repeat** The loop repeats many times.

Key takeaway:
Training changes weights; ordinary inference does not.

Training Loop

Broad Pretraining

Scale, not perfect recall

Many examples flow through the training loop. Repeated updates shape weights into broad patterns the model can use later. This does not make the model a perfect memory of its sources.

- **Many examples flow** Pretraining repeats the training loop across enormous collections of examples.
- **Updates shape weights** Prediction errors drive repeated weight updates during training.
- **Broad patterns form** The model learns statistical patterns that can help later prompts.
- **Not perfect recall** Pretraining does not store every source as a searchable memory.

Key takeaway:
Pretraining builds broad capability, not perfect source recall.

Broad Pretraining

Overfitting vs Generalization

Memorizing training examples is not the same as learning patterns that transfer to set-aside validation examples.

1. **Training examples** Old dots are examples the model fit during training.
2. **Validation examples** Set-aside examples are kept out of training to test transfer.
3. **Overfit curve** The overfit curve traces old dots too tightly.
4. **Generalizing curve** The smoother curve reaches new examples better.

Key takeaway:
A model should learn transferable patterns, not just memorize the training dots.

Overfitting vs Generalization

Fine-Tuning Path

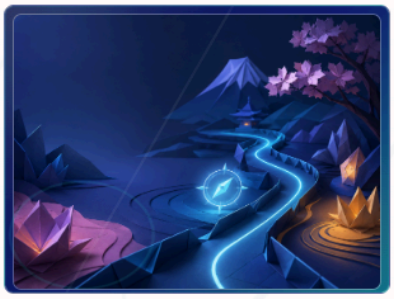
Durable adaptation, not current-run steering

Fine-tuning can durably adapt model behavior through additional training or adapters; prompts, RAG, and sampling steer only the current run.

- **Durable adaptation** Additional training or adapters can change how future runs behave.
- **Prompt steering** A prompt can steer this run without changing weights.
- **RAG and context** Retrieved text can enter the current context without training the model.
- **Sampling later** Decoding still chooses one next token from probabilities during inference.

Key takeaway:
Fine-tuning changes future behavior; prompting, RAG, and sampling operate during the current run.

Fine-Tuning Path



Alignment Landscape

Shaping behavior, not conscience

Alignment encourages preferred behavior, uses guardrails to reduce risky paths, and relies on feedback and policies without giving the model a conscience.

- **Preferred behavior** Training and system design can encourage more useful response patterns.
- **Guardrails** Runtime safeguards can reduce risky paths during use.
- **Feedback and policies** Human feedback, evaluations, policies, and filters can shape behavior.
- **No conscience** Alignment does not give the model moral understanding or a truth guarantee.

Key takeaway:
Alignment shapes behavior, but it is not magic morality.

Alignment Landscape

Forward Pass

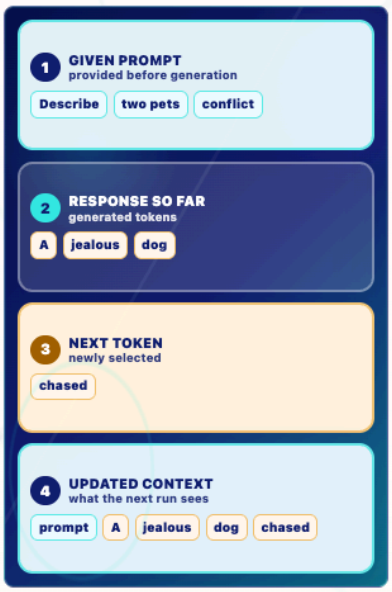
Fixed weights, temporary activations

Inference uses fixed weights to create temporary activations, hidden states, and next-token scores.

1. **Context** The current prompt, retrieved text, conversation, and response-so-far enter the run.
2. **Fixed weights** The learned weights are used but not normally updated.
3. **Temporary activations** The run creates temporary internal states that lead to next-token scores.
4. **Next token** Decoding chooses one response token, then the loop can repeat.

Key takeaway:
Inference is a forward pass, not durable training.

Forward Pass



Prompt vs Response

Given context versus generated tokens

Prompt tokens are given to the model; response tokens are generated one at a time and appended to the next context.

1. **Given prompt** The request "Describe two pets in conflict" is already provided input.
2. **Generated response** The response-so-far "A jealous dog" was generated by the model.
3. **Next token** The newly selected token "chased" is appended to the response.
4. **Updated context** The next run sees prompt plus response so far plus the new token.

> **Key takeaway:**
> Prompt is given; response tokens are generated and appended.

Prompt vs Response

Text to Tokens

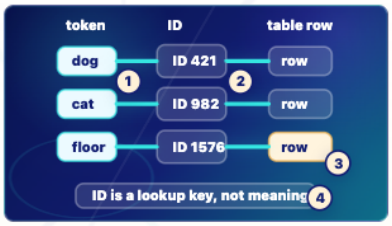
Uneven chunks, punctuation included

Text is split into model-readable chunks before token IDs and embedding lookup.

1. **Simplified split** A | jealous | dog | chased | a | startled | cat | across | the | kitchen | floor | .
2. **Uneven chunks** Some tokenizers may split words or punctuation, such as start | led or floor | .
3. **Teaching note** This app uses a simplified split; real tokenizer output can differ.

> **Key takeaway:**
> Tokens are model-readable chunks, not always human words.

Text to Tokens



Token IDs

Lookup keys, not meaning

Each token gets a lookup number that points to an embedding-table row.

1. **Token** dog, cat, and floor are text chunks.
2. **ID** 421, 982, and 1576 are lookup keys.
3. **Embedding row** The ID points to the learned starting vector row.
4. **Limit** The number itself is not the meaning.

> **Key takeaway:**
> Token IDs point; they do not understand.

Token IDs

Human Testing Recommendation

ready for small human testing. No P0/P1 visual remains after the readiness classification. Several visuals remain intentionally temporary and should be watched closely during testing.